

Contrato Técnico do Sistema CanGuess

Versão: 0.2 Status: DRAFT Modelo: Event-Sourced Knowledge Contract (Adaptado) Última atualização: 2026-06-11

1. Propósito

Este documento define o contrato operacional e arquitetural do sistema **CanGuess**, um sistema de bolão online. Ele serve como a fonte de verdade para o desenvolvimento, manutenção e evolução do sistema, garantindo que todos os agentes (humanos e artificiais) compreendam a estrutura de dados, fluxos de negócio, convenções e comportamentos esperados. O objetivo é minimizar retrocessos e garantir a consistência arquitetural.

2. Princípios Fundamentais

O sistema CanGuess adota um modelo de **Event Sourcing de Conhecimento**, conforme descrito no modelo inicial. Isso implica:

- Não substituição de conhecimento:** Informações e regras não são simplesmente sobrescritas, mas evoluem.
- Evolução incremental por blocos:** O sistema é construído e modificado em unidades lógicas e rastreáveis.
- Preservação de versões anteriores:** Versões passadas de regras e dados são mantidas para histórico e auditoria.

As regras e componentes do sistema podem ter os seguintes estados:

- ACTIVE:** Em uso e totalmente funcional.
- DEPRECATED:** Mantido para histórico, mas não recomendado para novo desenvolvimento. Pode ser removido em futuras versões.
- OVERRIDDEN:** Substituído por uma versão mais nova ou alternativa.

3. Convenções de Banco de Dados (Supabase/PostgreSQL)

O banco de dados do CanGuess é implementado no Supabase, utilizando PostgreSQL. As seguintes convenções são observadas:

3.1 Identidade Primária

Todas as tabelas utilizam `id` como chave primária (PK), tipicamente um UUID. Não são utilizados campos genéricos `*_id` para relações internas.

3.2 Convenção de Relacionamentos (Chaves Estrangeiras)

Para referenciar entidades, são utilizados campos com o sufixo `_uuid`, garantindo clareza e evitando ambiguidades. A tabela a seguir ilustra as convenções de relacionamento:

Entidade Origem	Campo de Chave Estrangeira	Entidade Destino
<code>event_phases</code>	<code>event_uuid</code>	<code>events (id)</code>
<code>event_rounds</code>	<code>event_phase_uuid</code>	<code>event_phases (id)</code>
<code>event_rounds</code>	<code>event_uuid</code>	<code>events (id)</code>
<code>event_parts</code>	<code>round_uuid</code>	<code>event_rounds (id)</code>
<code>event_passw</code>	<code>event_uuid</code>	<code>events (id)</code>
<code>guesses</code>	<code>event_round_id</code>	<code>rounds (id)</code>
<code>guesses</code>	<code>event_uuid</code>	<code>events (id)</code>
<code>regions</code>	<code>country_id</code>	<code>countries (id)</code>
<code>cities</code>	<code>region_id</code>	<code>regions (id)</code>

3.3 Regra Crítica de Nomenclatura

- ❌ **Nunca usar:** `phase_id`, `event_id`, `round_id` como nomes de colunas para chaves estrangeiras.
- ✅ **Sempre usar:** `phase_uuid`, `event_uuid`, `event_phase_uuid`, `round_uuid` (ou similar, quando referenciando semanticamente o UUID da entidade).

4. Entidades Principais do Sistema

As principais tabelas e suas estruturas de colunas são detalhadas abaixo. Note que `id` é sempre um UUID e PK.

4.1 events

Representa um evento principal (ex: Copa do Mundo, Campeonato Brasileiro).

- `id` (uuid)
- `name` (text)
- `description` (text)
- `event_type_uuid` (uuid, FK para `event_types.id`)
- `num_phases` (integer)
- `data_inicio` (date)
- `data_fim` (date)
- `created_at` (timestamp with time zone)

4.2 event_phases

Representa uma fase dentro de um evento (ex: Fase de Grupos, Oitavas de Final).

- `id` (uuid)
- `event_uuid` (uuid, FK para `events.id`)
- `phase_name` (text)
- `num_rounds` (integer)
- `phase_number` (integer, implicitamente usado para ordenação)

4.3 event_rounds

Representa uma rodada específica dentro de uma fase (ex: Rodada 1 da Fase de Grupos).

- `id` (uuid)
- `event_uuid` (uuid, FK para `events.id`)
- `event_phase_uuid` (uuid, FK para `event_phases.id`)
- `round_number` (integer)
- `round_name` (text, opcional)
- `date` (date, opcional)
- `time` (time, opcional)
- `location` (text, opcional)

4.4 event_parts

Representa as partes individuais de um palpite dentro de uma rodada (ex: para um placar 2x1, '2' e '1' seriam parts).

- `id` (uuid)
- `round_uuid` (uuid, FK para `event_rounds.id`)
- `part_number` (integer)
- `value` (text, pode ser nulo)

4.5 event_types

Define os tipos de eventos (ex: `team_vs_team`, `individual`, `metric`).

- `id` (uuid)
- `name` (text)

4.6 event_passw

Utilizado para validar o acesso de usuários a um evento específico.

- `code_event` (text)
- `event_uuid` (uuid, FK para `events.id`)
- `status` (integer, 1 para ativo, 0 para usado)

4.7 guesses

Armazena os palpites dos usuários.

- `id` (uuid)
- `user_uuid` (uuid, ou placeholder 'anonymous')
- `event_round_id` (uuid, FK para `rounds.id` - **Nota: Discrepância com event_rounds**)
- `event_guess` (text)
- `event_uuid` (uuid, FK para `events.id`)

4.8 rounds (Discrepância Arquitetural)

Esta tabela foi identificada no `Event.jsx` para carregar rounds para o usuário final. Sua relação com `event_rounds` não é explícita e parece ser uma discrepância arquitetural ou um

legado. O `Event.jsx` carrega `rounds` ordenados por `round_date` e os associa a `event_uuid`.

- `id` (uuid)
- `round_date` (date)
- `event_uuid` (uuid, FK para `events.id`)
- `round_order` (integer, opcional)

4.9 Entidades Auxiliares (Times e Localidades)

O sistema também possui tabelas para gerenciar times e suas localidades, usadas principalmente em cadastros.

- `countries` (`id`, `code`, `country`)
 - `regions` (`id`, `country_id`, `code`, `region`)
 - `cities` (`id`, `region_id`, `name`)
 - `times` (`id`, `nome`, `codigo`, `divisao`, `cidade`)
 - `round_games` (`id`, `home_team`, `away_team`, `game_date`, `local`) - **Nota: Sem ligação direta com `events`**
-

5. RPCs (Remote Procedure Calls) do Supabase

As RPCs são funções PostgreSQL que encapsulam a lógica de negócio e garantem a consistência dos dados. Elas são a forma preferencial de mutação de dados no sistema.

5.1 `create_default_phases(p_event_id uuid, p_num_phases integer)`

- **Propósito:** Criar um número `p_num_phases` de fases padrão para um evento recém-criado.
- **Uso:** Invocada após a inserção de um novo evento na tabela `events` (ex: `CadastrosEventos.jsx`).
- **Comportamento:** Insere `p_num_phases` registros na tabela `event_phases`, associando-os ao `p_event_id` e definindo `phase_name` como 'Fase X' e `num_rounds` como 0.
- **Retorno:** `text` (ex: 'SUCESSO').

5.2 `add_event_phases(p_event_id uuid, p_current_phases integer, p_new_phases integer)`

- **Propósito:** Adicionar fases a um evento existente quando o número total de fases é aumentado.
- **Uso:** Invocada a partir da UI de edição de eventos (ex: `CadastrosEventos.jsx`) quando `num_phases` é incrementado.
- **Comportamento:** Insere `p_new_phases - p_current_phases` novas fases na tabela `event_phases` para o `p_event_id`.
- **Retorno:** `text` (ex: 'SUCESSO').

5.3 `add_event_rounds(p_phase_id uuid, p_new_rounds integer)` (DEPRECATED/OVERRIDDEN)

- **Propósito Original:** Adicionar rounds a uma fase, garantindo que apenas a diferença necessária seja inserida e que o número de rounds não seja reduzido.
- **Status:** Esta função é considerada **DEPRECATED** e **OVERRIDDEN** pela `t_add_event_rounds`. O modelo inicial a descreve, mas o código frontend (`CadastrosFases.jsx`) utiliza a versão `t_add_event_rounds`.
- **Comportamento Original:** Contava os rounds existentes, verificava se `p_new_rounds` era maior e inseria a diferença. Impedia a redução de rounds.
- **Retorno:** `text` (ex: 'SUCESSO', 'AVISO', 'ERRO').

5.4 `t_add_event_rounds(p_phase_id uuid, p_num_rounds integer)` (ACTIVE)

- **Propósito:** Gerenciar a criação e sincronização de rounds para uma fase específica de forma robusta, incluindo a limpeza de dependências.
- **Uso:** Invocada pela UI de gerenciamento de fases (`CadastrosFases.jsx`) para criar ou atualizar o número de rounds de uma fase.
- **Comportamento:**
 1. Obtém o `event_uuid` da fase.
 2. **Deleta** `event_parts` associados aos rounds da fase para evitar erros de chave estrangeira.
 3. **Deleta todos os** `event_rounds` existentes para a `p_phase_id`.
 4. **Recria** `p_num_rounds` novos registros em `event_rounds` para a `p_phase_id`, com `round_number` de 1 a `p_num_rounds`.

- **Implicação:** Esta função sobrescreve completamente os rounds de uma fase, o que significa que qualquer edição manual ou dados associados a rounds individuais seriam perdidos se não forem recriados. A deleção em cascata de `event_parts` é uma característica importante.
 - **Retorno:** `text` (ex: 'SUCESSO').
-

6. Fluxos do Sistema (UI → Backend → RPC → Persistência)

6.1 Criação de Evento

- **Origem:** Frontend - `CadastrosEventos.jsx`
- **Ação:** O usuário preenche o formulário de novo evento e clica em salvar.
- **Persistência:** Um `INSERT` é realizado na tabela `events` com os dados fornecidos pelo usuário.
- **Pós-criação:** Após o sucesso do `INSERT`, a RPC `create_default_phases(p_event_id, p_num_phases)` é invocada para gerar as fases iniciais do evento.
- **Regra:** A RPC cria `N` fases automaticamente, onde `N` é o valor de `num_phases` fornecido, e cada fase inicia com `num_rounds = 0`.

6.2 Criação e Edição de Fases

- **Origem:** Frontend - `CadastrosFases.jsx`
- **Ação:** O usuário acessa a tela de fases de um evento. Pode adicionar novas fases ou editar fases existentes (nome e número de rounds).
- **Persistência (Fases Existentes):** Para fases existentes, um `UPDATE` é realizado na tabela `event_phases` para `phase_name` e `num_rounds`.
- **Persistência (Novas Fases):** Para novas fases adicionadas via UI, um `INSERT` é realizado na tabela `event_phases`.
- **Sincronização de Rounds:** Após a atualização ou inserção de uma fase, a RPC `t_add_event_rounds(p_phase_id, p_num_rounds)` é invocada. Esta RPC é responsável por garantir que o número correto de rounds exista para a fase, deletando e recriando-os conforme necessário (ver seção 5.4).

6.3 Sincronização de Rounds (Regra Central)

Este fluxo descreve a lógica de negócio para gerenciar o número de rounds em uma fase, conforme observado no `CadastrrosFases.jsx` e `simple_rpc_v2.sql`.

- **Origem:** Frontend - `CadastrrosFases.jsx` (botão “Salvar”).
- **Entrada da UI:** O usuário define o valor `newRounds` para uma fase.
- **Regra de Decisão (Executada no Backend via RPC):** A lógica de sincronização foi migrada para a RPC `t_add_event_rounds`.
 - A RPC recebe o `phase_id` e o `num_rounds` desejado.
 - A RPC **deleta** todos os `event_parts` associados aos rounds da fase.
 - A RPC **deleta** todos os `event_rounds` da fase.
 - A RPC **recria** `num_rounds` registros em `event_rounds`.
- **Nota de Desvio Arquitetural:** O modelo inicial sugeria que a UI validaria se `new_rounds < current_rounds` e bloquearia a redução, e que a RPC `add_event_rounds` apenas inseriria a diferença. A implementação atual (`t_add_event_rounds`) permite a redução e recria todos os rounds, o que é uma abordagem mais destrutiva, mas simplifica a lógica de sincronização.

6.4 Geração de Parts

- **Origem:** Frontend - `CadastrrosParts.jsx`
- **Ação:** O usuário seleciona um evento, fase e round, e clica em “Gerar Parts Automático” ou “Criar Part”.
- **Regra (Automático):** O sistema verifica o número de parts existentes para o round e o número esperado (`round.event_parts_count` ou 1). Se houver menos parts do que o esperado, ele insere a diferença na tabela `event_parts`.
- **Regra (Manual):** O sistema insere uma nova part na tabela `event_parts`, desde que o número total não exceda `round.event_parts_count`.

6.5 Submissão de Palpites (Consumer Flow)

- **Origem:** Frontend - `Event.jsx`
- **Ação:** O usuário insere o código do evento para acessá-lo. O sistema valida o código na tabela `event_passw`.
- **Carregamento:** O sistema carrega os dados do evento, fases e rounds (da tabela `rounds`, não `event_rounds`).

- **Persistência:** Quando o usuário insere um palpite, um `INSERT` é realizado na tabela `guesses` com o `user_uuid` (atualmente 'anonymous'), `event_round_id`, `event_guess` e `event_uuid`.
-

7. Regras de Consistência e Triggers

7.1 Triggers (Legado/Opcional)

Triggers são considerados **DEPRECATED** no modelo atual. A lógica de negócio, como a criação de fases e rounds, foi migrada para RPCs determinísticas (ex: `create_default_phases`, `t_add_event_rounds`). Isso evita side-effects invisíveis e centraliza a lógica em funções explícitas.

7.2 Regras de Consistência

- **Proibido:**
 - Inferir IDs diferentes dos definidos no schema (ex: usar `phase_id` em vez de `phase_uuid`).
 - Confiar em triggers não documentados para lógica de negócio.
 - A UI escrever diretamente em tabelas dependentes (ex: `event_rounds`) em cascata. A UI deve invocar RPCs para operações complexas.
 - **Obrigatório:**
 - Toda mutação de dados deve ter uma origem explícita: ação da UI, chamada de RPC ou função server-side.
 - A UI deve enviar a intenção (ex: `new_rounds`) e a identidade (`phase_uuid`) para o backend, que executa a lógica de consistência.
-

8. Desvios Arquiteturais (Drift)

Durante a análise do sistema, foram identificados os seguintes desvios em relação ao modelo inicial:

1. **Sincronização de Rounds:** O modelo inicial previa a RPC `add_event_rounds` que apenas adicionava rounds faltantes e impedia a redução. A implementação atual utiliza `t_add_event_rounds`, que deleta e recria todos os rounds e parts associados a uma fase, permitindo a redução, mas com comportamento destrutivo.
-

2. **Tabela rounds vs event_rounds** : O fluxo de administração (`CadastrosFases.jsx` , `CadastrosRounds.jsx`) utiliza a tabela `event_rounds` . No entanto, o fluxo do consumidor (`Event.jsx`) carrega dados da tabela `rounds` e salva palpites referenciando `rounds.id` . Esta é uma discrepância significativa que precisa ser resolvida para garantir a consistência do sistema.
 3. **Tabela round_games** : O arquivo `CadastrosRodadas.jsx` gerencia jogos em uma tabela `round_games` , que não parece estar integrada com a hierarquia principal de eventos (`events` -> `event_phases` -> `event_rounds`).
-

9. Resumo Operacional

- **Events** criam **Phases** (via RPC `create_default_phases`).
 - **Phases** definem **Rounds** (via RPC `t_add_event_rounds`).
 - **Rounds** contêm **Parts** (gerados via UI em `CadastrosParts.jsx`).
 - A UI não escreve diretamente em cascata; o banco de dados executa a consistência lógica através de RPCs.
 - O sistema utiliza um modelo de Event Sourcing de Conhecimento, priorizando a evolução e preservação de regras.
-

Fim do Contrato

Este documento representa o estado atual e o contrato arquitetural do sistema CanGuess. Qualquer divergência entre o código e este contrato deve ser considerada um **DRIFT DE ARQUITETURA** e tratada adequadamente.